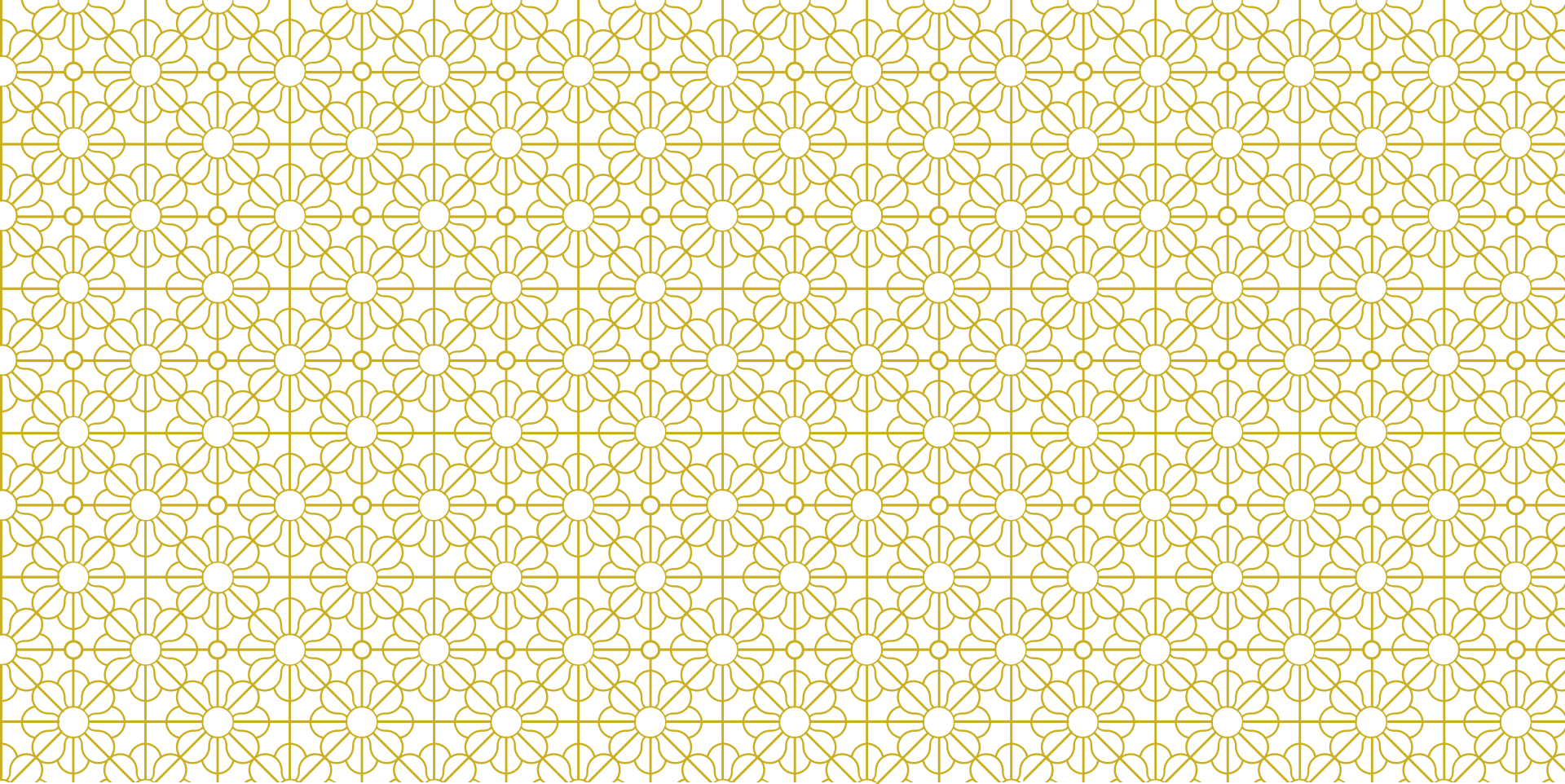
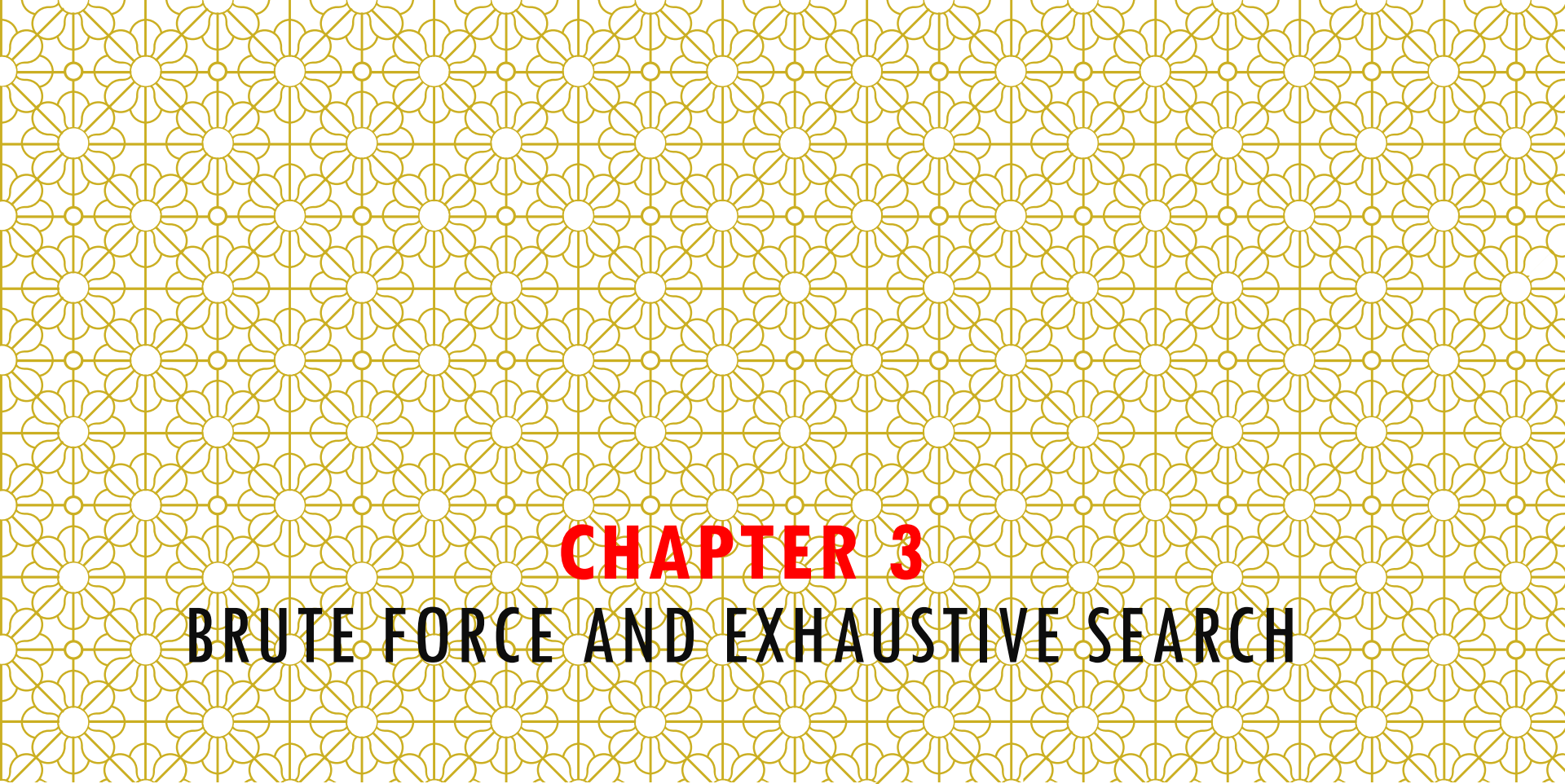




# **ALGORITHMS ANALYSIS AND DESIGN**

By : Dr. Basem  
Alijla



# **CHAPTER 3**

## **BRUTE FORCE AND EXHAUSTIVE SEARCH**

# OUTLINE

- **Introduction to Brute-Force approach**
- **Selection Sort and Bubble Sort**
- **Sequential Search and Brute-Force String Matching**
- **Closest-Pair Problem by Brute Force**
- **Exhaustive Search**
- **Depth-First Search and Breadth-First Search**

# BRUTE FORCE AND EXHAUSTIVE SEARCH

## **Brute Force :**

A straightforward approach, usually based directly on the problem's statement and definitions of the concepts involved.

Computer Force not one's intellect

**“ Just Do It “**

# BRUTE FORCE ALGORITHM TO FIND THE GCD

## Consecutive integer checking algorithm

**Step 1** Assign the value of  $\min(m, n)$  to  $t$

**Step 2** Divide  $m$  by  $t$ . If the remainder is 0, go to Step 3;  
otherwise, go to Step 4

**Step 3** Divide  $n$  by  $t$ . If the remainder is 0, return  $t$  and stop;  
otherwise, go to Step 4

**Step 4** Decrease  $t$  by 1 and go to Step 2  $O(n)$

## BRUTE FORCE ALGORITHM TO COMPUTE $A^N$

compute  $a^n$  for a nonzero number  $a$  and a nonnegative integer  $n$ .

Multiply 1 by  $a$   $n$  times  $a^n = \underbrace{a \times a \times \dots \times a}_n$

**P = 1 ; I = 0 ;**

**While ( I < n ) {**

**p = p \* a ;**

**I ++ ;**

**}**

**$O(n)$**

# BRUTE FORCE ALGORITHM TO FIND A KEY IN LIST

**Searching for a key of a given value in a list**

Matched = false ;

While ( i < n && ! matched ) {

    if( list ( i ) == key )

        matched = true;

    i++ ;

}

$O(n)$

# OUTLINE

- **Introduction to Brute-Force approach**
- **Selection Sort and Bubble Sort**
- **Sequential Search and Brute-Force String Matching**
- **Closest-Pair Problem by Brute Force**
- **Exhaustive Search**
- **Depth-First Search and Breadth-First Search**

# SELECTION SORT ALGORITHM

- Scan the array to find its smallest element, and swap it with the first element.
- Scan the elements to the right of it to find the second smallest among them and swap it with the second elements.
- on pass  $i$  ( $0 \leq i \leq n-2$ ), find the smallest element in  $A[i..n-1]$  and swap it with  $A[i]$ :

$$A[0] \leq \dots \leq A[i-1] \mid A[i], \dots, A[\text{min}], \dots, A[n-1]$$

↑
in their final positions
↑

# SELECTION SORT: BRUTE-FORCE SORTING ALGORITHM

Example: 89, 45, 68, 90, 29, 34, 17

| step: | Position                                                                                                       | Smallest |
|-------|----------------------------------------------------------------------------------------------------------------|----------|
| (a)   | (89,45, 68,90,29, 34,17)<br> | (17)     |
| (b)   | ( <b>17</b>  45, 68, 90,29, 34,89)                                                                             | (29)     |
| (c)   | ( <b>17, 29</b> / <del>68</del> , 90,45,34,89)                                                                 | (34)     |
| (d)   | ( <b>17, 29, 34</b> / <del>90</del> ,45, 68,89)                                                                | (45)     |
| (e)   | ( <b>17, 29, 34, 45</b> / <del>90</del> ,68,89)                                                                | (68)     |
| (f)   | ( <b>17, 29, 34, 45, 68</b> / <del>90</del> ,89)                                                               | (89)     |
| (g)   | ( <b>17, 29, 34, 45, 68,89</b>  90)                                                                            | stop     |

# ANALYSIS OF SELECTION SORT

**ALGORITHM** *SelectionSort*( $A[0..n - 1]$ )

//Sorts a given array by selection sort

//Input: An array  $A[0..n - 1]$  of orderable elements

//Output: Array  $A[0..n - 1]$  sorted in ascending order

**for**  $i \leftarrow 0$  **to**  $n - 2$  **do**

$min \leftarrow i$

**for**  $j \leftarrow i + 1$  **to**  $n - 1$  **do**

**if**  $A[j] < A[min]$   $min \leftarrow j$

    swap  $A[i]$  and  $A[min]$

Time efficiency :  $O(n^2)$

Space efficiency :  $O(n)$

# BUBBLE SORT: BRUTE-FORCE SORTING ALGORITHM

compare adjacent elements of the list and exchange them if they are out of order.

By doing it repeatedly, “bubbling up” the largest element to the last position on the list.

The next pass bubbles up the second largest element, and so on, until after  $n - 1$  passes the list is sorted.

Pass  $i$  ( $0 \leq i \leq n - 2$ ) of bubble sort can be represented by the following diagram.



## BUBBLE SORT: BRUTE-FORCE SORTING ALGORITHM

Example: 89, 45, 68, 90, 29, 34, 17

Turn 1

| step: | Position               |                      |
|-------|------------------------|----------------------|
| (a)   | (89,45,68,90,29,34,17) | Swap(89,45)          |
| (b)   | (45,89,68,90,29,34,17) | Swap(89,68)          |
| (c)   | (45,68,89,90,29,34,17) | no swap              |
| (d)   | (45,68,89,90,29,34,17) | Swap(90,29)          |
| (e)   | (45,68,89,29,90,34,17) | Swap(90,34)          |
| (f)   | (45,68,89,29,34,90,17) | Swap(90,19)          |
| (g)   | (45,68,89,29,34,17 90) | 90 at right position |

## BUBBLE SORT: BRUTE-FORCE SORTING ALGORITHM

Turn 2

| step: | Position                                    |
|-------|---------------------------------------------|
| (a)   | (45,68,89,29,34,17 90) no swap              |
| (b)   | (45,68,89,29,34,17 90) no swap              |
| (c)   | (45,68,89,29,34,17 90) Swap(89,29)          |
| (d)   | (45,68,29,89,34,17 90) Swap(89,34)          |
| (e)   | (45,68,29,34,89,17 90) Swap(89,17)          |
| (f)   | (45,68,29,34,17 89,90) 89 at right position |

Continue until no swap

(17,29,34,45,68,89,90)

## ANALYSIS OF BUBBLE SORT

**ALGORITHM** *BubbleSort*( $A[0..n - 1]$ )

//Sorts a given array by bubble sort

//Input: An array  $A[0..n - 1]$  of orderable elements

//Output: Array  $A[0..n - 1]$  sorted in nondecreasing order

**for**  $i \leftarrow 0$  **to**  $n - 2$  **do**

**for**  $j \leftarrow 0$  **to**  $n - 2 - i$  **do**

**if**  $A[j + 1] < A[j]$  **swap**  $A[j]$  and  $A[j + 1]$

Time efficiency :  $O(n^2)$

Space efficiency :  $O(n)$

# OUTLINE

- **Introduction to Brute-Force approach**
- **Selection Sort and Bubble Sort**
- **Sequential Search and Brute-Force String Matching**
- **Closest-Pair Problem by Brute Force**
- **Exhaustive Search**
- **Depth-First Search and Breadth-First Search**

# SEQUENTIAL SEARCH

**ALGORITHM** *SequentialSearch2*( $A[0..n]$ ,  $K$ )

//Implements sequential search with a search key as a sentinel

//Input: An array  $A$  of  $n$  elements and a search key  $K$

//Output: The index of the first element in  $A[0..n - 1]$  whose value is

// equal to  $K$  or  $-1$  if no such element is found

$A[n] \leftarrow K$

$i \leftarrow 0$

**while**  $A[i] \neq K$  **do**

$i \leftarrow i + 1$

**if**  $i < n$  **return**  $i$

**else return**  $-1$

Time efficiency :

Space efficiency :

# BRUTE-FORCE STRING MATCHING

pattern: a string of  $m$  characters to search for.

Text: a (longer) string of  $n$  characters to search in.

Problem: find a substring in the **text** that matches the **pattern** ( $m < n$ ).

Find the  $i$ -index of the leftmost character of the first matching substring

Examples :

Text: 10010101101001100101111010

Pattern: 001011

Pattern: happy

Text: It is never too late to have a happy childhood

## BRUTE-FORCE STRING MATCHING

10010101101001100101111010  
001011

10010101101001100101111010  
001011

10010101101001100101111010  
001011

10010101101001100101111010  
001011

10010101101001100101111010  
001011

10010101101001100101111010  
001011

# BRUTE-FORCE STRING MATCHING

**ALGORITHM** *BruteForceStringMatch*( $T[0..n - 1]$ ,  $P[0..m - 1]$ )

//Implements brute-force string matching

//Input: An array  $T[0..n - 1]$  of  $n$  characters representing a text and

// an array  $P[0..m - 1]$  of  $m$  characters representing a pattern

//Output: The index of the first character in the text that starts a

// matching substring or  $-1$  if the search is unsuccessful

**for**  $i \leftarrow 0$  **to**  $n - m$  **do**

$j \leftarrow 0$

**while**  $j < m$  **and**  $P[j] = T[i + j]$  **do**

$j \leftarrow j + 1$

**if**  $j = m$  **return**  $i$

**return**  $-1$

Efficiency:  $O(nm)$

# ADVANTAGES OF BRUTE FORCE STRATEGY

- General approach, wide applicability
- Brute-force approach yields reasonable algorithms of at least some practical value with no limitation on instance size.
- The expense of designing a more efficient algorithm may be unjustifiable.
- Useful for solving small-size instances of a problem.
- Serve as a metric to judge more efficient alternatives

# DISADVANTAGES OF BRUTE FORCE STRATEGY

- rarely yields efficient algorithms
- some brute-force algorithms are unacceptably slow
- not as constructive as some other design techniques